Suffix Arrays II

Paweł Gawrychowski

University of Wrocław

Recall that we assume the existence of the following device:

Suffix array + constant time RMQ queries

Given a text $w[1..n]$, we can construct in linear time a structure of size $\mathcal{O}(n)$ which allows us to answer any query of the form "what is the longest common prefix of $w[i..n]$ and $w[j..n]$?" in constant time.

During the next two lectures we will see how this can be done.

Recall that we assume the existence of the following device:

Suffix array + constant time RMQ queries

Given a text $w[1..n]$, we can construct in linear time a structure of size $\mathcal{O}(n)$ which allows us to answer any query of the form "what is the longest common prefix of $w[i..n]$ and $w[j..n]$?" in constant time.

During the next two lectures we will see how this can be done.

Recall that we assume the existence of the following device:

Suffix array + constant time RMQ queries

Given a text $w[1..n]$, we can construct in linear time a structure of size $\mathcal{O}(n)$ which allows us to answer any query of the form "what is the longest common prefix of $w[i..n]$ and $w[j..n]$?" in constant time.

During the next two lectures we will see how this can be done.

Goal

We already know how to solve the pattern matching problems: given a pattern and a text we can find all occurrences in linear time. But what if the text doesn't change, and we need to answer **MANY** queries about the occurrences of different patterns?

This is known as text indexing, and can be solved using suffix arrays. Later we will see other solutions, which can be modified to work even for dynamically changing text (think about appending characters to a very long document...).

Goal

We already know how to solve the pattern matching problems: given a pattern and a text we can find all occurrences in linear time. But what if the text doesn't change, and we need to answer **MANY** queries about the occurrences of different patterns?

This is known as text indexing, and can be solved using suffix arrays. Later we will see other solutions, which can be modified to work even for dynamically changing text (think about appending characters to a very long document...).

Lexicographical comparison

$s \leq t$ if either s is a prefix of t, or s and t agree on the first $i-1$ positions, i.e., $s[1] = t[1]$, $s[2] = t[2]$, ..., $s[i-1] = t[i-1]$, and then $s[i] < t[i]$.

Compare *bbab* with *bbb*. Then compare *aab* with *aab*.

One can check that using the character $\leq$ makes sense: the relation is a total order (verify this!).

Lexicographical comparison

$s \leq t$ if either s is a prefix of t, or s and t agree on the first $i-1$ positions, i.e., $s[1] = t[1]$, $s[2] = t[2]$, ..., $s[i-1] = t[i-1]$, and then $s[i] < t[i]$.

Compare *bbab* with *bbb*. Then compare *aab* with *aab*.

One can check that using the character $\leq$ makes sense: the relation is a total order (verify this!).

Lexicographical comparison

$s \leq t$ if either s is a prefix of t, or s and t agree on the first $i-1$ positions, i.e., $s[1] = t[1]$, $s[2] = t[2]$, ..., $s[i-1] = t[i-1]$, and then $s[i] < t[i]$.

Compare *bbab* with *bbb*. Then compare *aab* with *aab*.

One can check that using the character $\leq$ makes sense: the relation is a total order (verify this!).

Now the suffix array is simply the lexicographically sorted list of all suffixes of a given word w.

$w =$ mississippi

$SA[1] = 11 =$ i
$SA[2] = 8 =$ ippi
$SA[3] = 5 =$ issippi
$SA[4] = 2 =$ ississippi
$SA[5] = 1 =$ mississippi
$SA[6] = 10 =$ pi
$SA[7] = 9 =$ ppi
$SA[8] = 7 =$ sippi
$SA[9] = 4 =$ sissippi
$SA[10] = 6 =$ ssippi
$SA[11] = 3 =$ ssissippi

Why this is useful?

Generating all occurrences of a given pattern

Say that we want to output all occurrences of $p = \texttt{ippi}$. How do they look when we look at the table?

Lemma

All occurrences of the same p create a contiguous fragment $SA[i], SA[i+1], \ldots, SA[j]$ of the suffix array.

Proof: look at the whiteboard.

The question is how to determine this fragment?

Why this is useful?

Generating all occurrences of a given pattern

Say that we want to output all occurrences of $p = \texttt{ippi}$. How do they look when we look at the table?

Lemma

All occurrences of the same p create a contiguous fragment $SA[i], SA[i+1], \ldots, SA[j]$ of the suffix array.

Proof: look at the whiteboard.

The question is how to determine this fragment?

Why this is useful?

Generating all occurrences of a given pattern

Say that we want to output all occurrences of $p = \texttt{ippi}$. How do they look when we look at the table?

Lemma

All occurrences of the same p create a contiguous fragment $SA[i], SA[i+1], \ldots, SA[j]$ of the suffix array.

Proof: look at the whiteboard.

The question is how to determine this fragment?

Why this is useful?

Generating all occurrences of a given pattern

Say that we want to output all occurrences of $p = \texttt{ippi}$. How do they look when we look at the table?

Lemma

All occurrences of the same p create a contiguous fragment $SA[i], SA[i+1], \ldots, SA[j]$ of the suffix array.

Proof: look at the whiteboard.

The question is how to determine this fragment?

Lemma

$SA[i-1] < p \leq SA[i]$.

Proof: look at the whiteboard.

Recall that $SA[1] < SA[2] < \ldots < SA[n]$. Hence we can binary search for the value of i! And then verify if it corresponds to an occurrence, i.e., whether p is indeed a prefix of $SA[i]$.

How to determine j?

Lemma

$SA[i-1] < p \leq SA[i]$.

Proof: look at the whiteboard.

Recall that $SA[1] < SA[2] < \ldots < SA[n]$. Hence we can binary search for the value of i! And then verify if it corresponds to an occurrence, i.e., whether p is indeed a prefix of $SA[i]$.

How to determine j?

Recall that our motivation for constructing the suffix array was that using it we can locate an occurrence of any pattern of length m in $\mathcal{O}(m \log n)$ time. Now we are almost ready to speed this up!

The suffix array SA alone is not that useful. Usually it is augmented with the inverse suffix array SA^{-1}, where $SA^{-1}[i]$ is the position of $w[i..n]$ in SA, i.e., $SA[SA^{-1}[i]] = i$, and with a longest common prefix structure.

LCP

$\mathrm{lcp}(i, j)$ is the longest common prefix of $w[i..n]$ and $w[j..n]$. $\mathrm{lcp}[i]$ is the longest common prefix of the $(i-1)$-th and i-th suffix in the suffix array, or in other words $\mathrm{lcp}(SA[i-1], SA[i])$, with $\mathrm{lcp}[1]$ not defined.

Recall that our motivation for constructing the suffix array was that using it we can locate an occurrence of any pattern of length m in $\mathcal{O}(m \log n)$ time. Now we are almost ready to speed this up!
The suffix array SA alone is not that useful. Usually it is augmented with the inverse suffix array SA^{-1}, where $SA^{-1}[i]$ is the position of $w[i..n]$ in SA, i.e., $SA[SA^{-1}[i]] = i$, and with a longest common prefix structure.

LCP

$\mathrm{lcp}(i, j)$ is the longest common prefix of $w[i..n]$ and $w[j..n]$. $\mathrm{lcp}[i]$ is the longest common prefix of the $(i-1)$-th and i-th suffix in the suffix array, or in other words $\mathrm{lcp}(SA[i-1], SA[i])$, with $\mathrm{lcp}[1]$ not defined.

Recall that our motivation for constructing the suffix array was that using it we can locate an occurrence of any pattern of length m in $\mathcal{O}(m \log n)$ time. Now we are almost ready to speed this up!
The suffix array SA alone is not that useful. Usually it is augmented with the inverse suffix array SA^{-1}, where $SA^{-1}[i]$ is the position of $w[i..n]$ in SA, i.e., $SA[SA^{-1}[i]] = i$, and with a longest common prefix structure.

LCP

$\mathrm{lcp}(i, j)$ is the longest common prefix of $w[i..n]$ and $w[j..n]$. $\mathrm{lcp}[i]$ is the longest common prefix of the $(i-1)$-th and i-th suffix in the suffix array, or in other words $\mathrm{lcp}(SA[i-1], SA[i])$, with $\mathrm{lcp}[1]$ not defined.

$w =$ mississippi

$SA[1] =$	11	$=$	i
$SA[2] =$	8	$=$	ippi
$SA[3] =$	5	$=$	issippi
$SA[4] =$	2	$=$	ississippi
$SA[5] =$	1	$=$	mississippi
$SA[6] =$	10	$=$	pi
$SA[7] =$	9	$=$	ppi
$SA[8] =$	7	$=$	sippi
$SA[9] =$	4	$=$	sissippi
$SA[10] =$	6	$=$	ssippi
$SA[11] =$	3	$=$	ssissippi

What is $\mathrm{lcp}(8, 2)$?

$w =$ `mississippi`

$SA[1] =$	11	$=$	`i`			
$SA[2] =$	8	$=$	`ippi`	$\mathrm{lcp}[2]$	$=$	1
$SA[3] =$	5	$=$	`issippi`	$\mathrm{lcp}[3]$	$=$	1
$SA[4] =$	2	$=$	`ississippi`	$\mathrm{lcp}[4]$	$=$	4
$SA[5] =$	1	$=$	`mississippi`	$\mathrm{lcp}[5]$	$=$	0
$SA[6] =$	10	$=$	`pi`	$\mathrm{lcp}[6]$	$=$	0
$SA[7] =$	9	$=$	`ppi`	$\mathrm{lcp}[7]$	$=$	1
$SA[8] =$	7	$=$	`sippi`	$\mathrm{lcp}[8]$	$=$	0
$SA[9] =$	4	$=$	`sissippi`	$\mathrm{lcp}[9]$	$=$	2
$SA[10] =$	6	$=$	`ssippi`	$\mathrm{lcp}[10]$	$=$	1
$SA[11] =$	3	$=$	`ssissippi`	$\mathrm{lcp}[11]$	$=$	3

What is $\mathrm{lcp}(8, 2)$?

Lemma

$\mathrm{lcp}(i, j)$ is the minimum of all $\mathrm{lcp}[k]$ over $k = SA^{-1}[i] + 1$, $SA^{-1}[i] + 2$, ..., $SA^{-1}[j]$, assuming i is before j in the suffix array.

Proof: see the whiteboard.

So what?

So, assuming that we know $\mathrm{lcp}[i]$, computing any $\mathrm{lcp}(i, j)$ requires just one range minimum query.

RMQ

Given an array $A[1..n]$, preprocess it so that the minimum of any fragment $A[i], A[i + 1], \ldots, A[j]$ can be computed efficiently.

During the next lecture, we will see a linear time/space preprocessing allowing answering any query in constant time.

Lemma

$\mathrm{lcp}(i, j)$ is the minimum of all $\mathrm{lcp}[k]$ over $k = SA^{-1}[i] + 1$, $SA^{-1}[i] + 2$, ..., $SA^{-1}[j]$, assuming i is before j in the suffix array.

Proof: see the whiteboard.

So what?

So, assuming that we know $\mathrm{lcp}[i]$, computing any $\mathrm{lcp}(i, j)$ requires just one range minimum query.

RMQ

Given an array $A[1..n]$, preprocess it so that the minimum of any fragment $A[i], A[i + 1], \ldots, A[j]$ can be computed efficiently.

During the next lecture, we will see a linear time/space preprocessing allowing answering any query in constant time.

Lemma

$\mathrm{lcp}(i, j)$ is the minimum of all $\mathrm{lcp}[k]$ over $k = SA^{-1}[i] + 1$, $SA^{-1}[i] + 2$, ..., $SA^{-1}[j]$, assuming i is before j in the suffix array.

Proof: see the whiteboard.

So what?

So, assuming that we know $\mathrm{lcp}[i]$, computing any $\mathrm{lcp}(i, j)$ requires just one range minimum query.

RMQ

Given an array $A[1..n]$, preprocess it so that the minimum of any fragment $A[i], A[i + 1], \ldots, A[j]$ can be computed efficiently.

During the next lecture, we will see a linear time/space preprocessing allowing answering any query in constant time.

Lemma

$\mathrm{lcp}(i, j)$ is the minimum of all $\mathrm{lcp}[k]$ over $k = SA^{-1}[i] + 1$, $SA^{-1}[i] + 2$, ..., $SA^{-1}[j]$, assuming i is before j in the suffix array.

Proof: see the whiteboard.

So what?

So, assuming that we know $\mathrm{lcp}[i]$, computing any $\mathrm{lcp}(i, j)$ requires just one range minimum query.

RMQ

Given an array $A[1..n]$, preprocess it so that the minimum of any fragment $A[i], A[i + 1], \ldots, A[j]$ can be computed efficiently.

During the next lecture, we will see a linear time/space preprocessing allowing answering any query in constant time.

OK, but how to compute the array $\mathrm{lcp}[2], \mathrm{lcp}[3], \ldots, \mathrm{lcp}[n]$?

A trivial solution is to compute each of them separately, resulting in $\mathcal{O}(n^2)$ total complexity. It turns out that a small modification to this naive method improves the time to linear.

Lemma

For any suffix $w[i..n]$, $\mathrm{lcp}[SA^{-1}[i]] - 1 \leq \mathrm{lcp}[SA^{-1}[i+1]]$, assuming $i < n$ and neither $SA^{-1}[i]$ nor $SA^{-1}[i+1]$ are the first element of the suffix array.

Kasai 2006

All lcp can be computed in amortized constant time per entry.

Proof: next problem set.

OK, but how to compute the array $\mathrm{lcp}[2], \mathrm{lcp}[3], \ldots, \mathrm{lcp}[n]$?

A trivial solution is to compute each of them separately, resulting in $\mathcal{O}(n^2)$ total complexity. It turns out that a small modification to this naive method improves the time to linear.

Lemma

For any suffix $w[i..n]$, $\mathrm{lcp}[SA^{-1}[i]] - 1 \leq \mathrm{lcp}[SA^{-1}[i+1]]$, assuming $i < n$ and neither $SA^{-1}[i]$ nor $SA^{-1}[i+1]$ are the first element of the suffix array.

Kasai 2006

All lcp can be computed in amortized constant time per entry.

Proof: next problem set.

OK, but how to compute the array $\mathrm{lcp}[2], \mathrm{lcp}[3], \ldots, \mathrm{lcp}[n]$?

A trivial solution is to compute each of them separately, resulting in $\mathcal{O}(n^2)$ total complexity. It turns out that a small modification to this naive method improves the time to linear.

Lemma

For any suffix $w[i..n]$, $\mathrm{lcp}[SA^{-1}[i]] - 1 \leq \mathrm{lcp}[SA^{-1}[i+1]]$, assuming $i < n$ and neither $SA^{-1}[i]$ nor $SA^{-1}[i+1]$ are the first element of the suffix array.

Kasai 2006

All lcp can be computed in amortized constant time per entry.

Proof: next problem set.

OK, but how to compute the array $\mathrm{lcp}[2], \mathrm{lcp}[3], \ldots, \mathrm{lcp}[n]$?

A trivial solution is to compute each of them separately, resulting in $\mathcal{O}(n^2)$ total complexity. It turns out that a small modification to this naive method improves the time to linear.

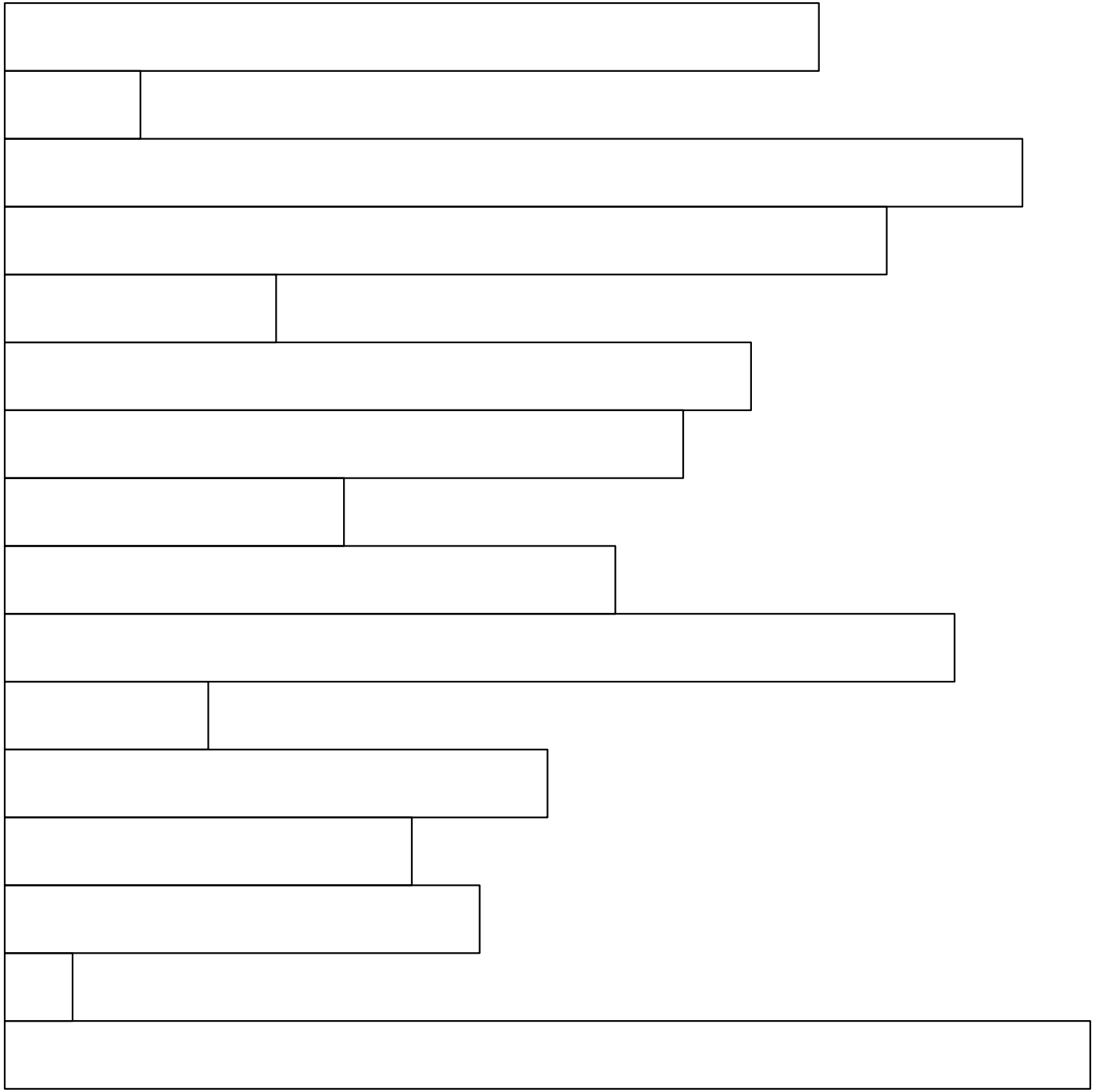
Lemma

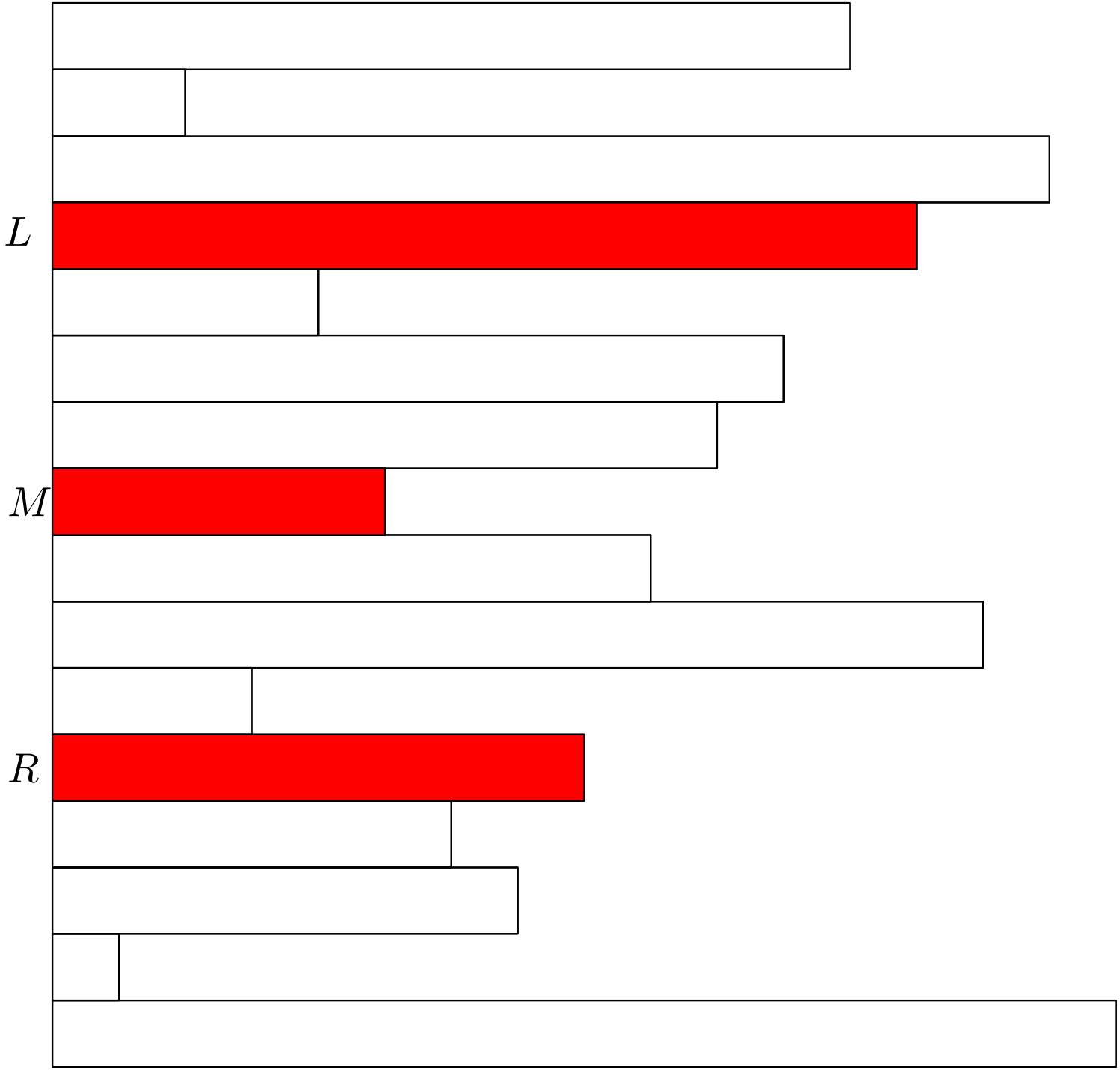
For any suffix $w[i..n]$, $\mathrm{lcp}[SA^{-1}[i]] - 1 \leq \mathrm{lcp}[SA^{-1}[i+1]]$, assuming $i < n$ and neither $SA^{-1}[i]$ nor $SA^{-1}[i+1]$ are the first element of the suffix array.

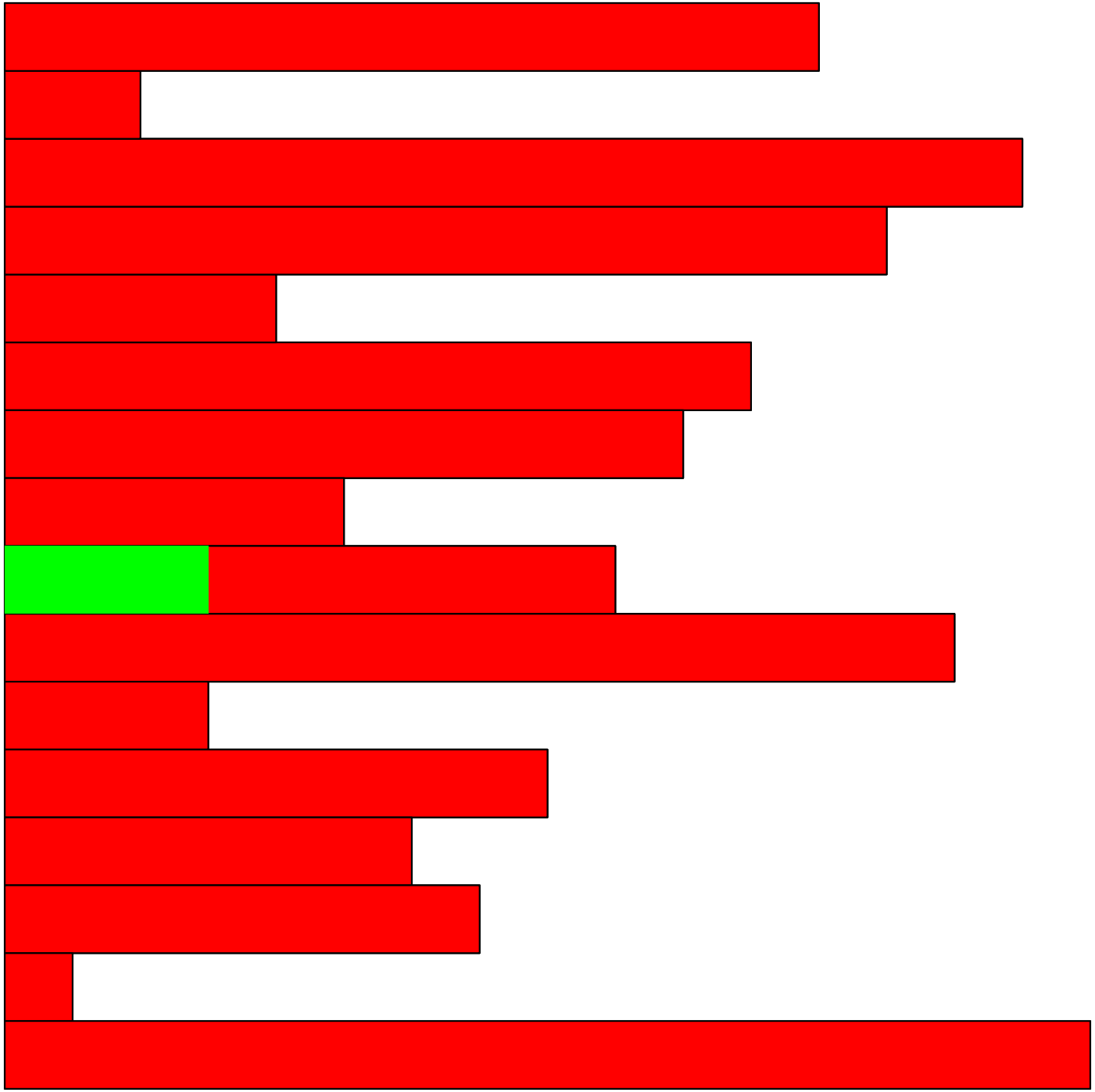
Kasai 2006

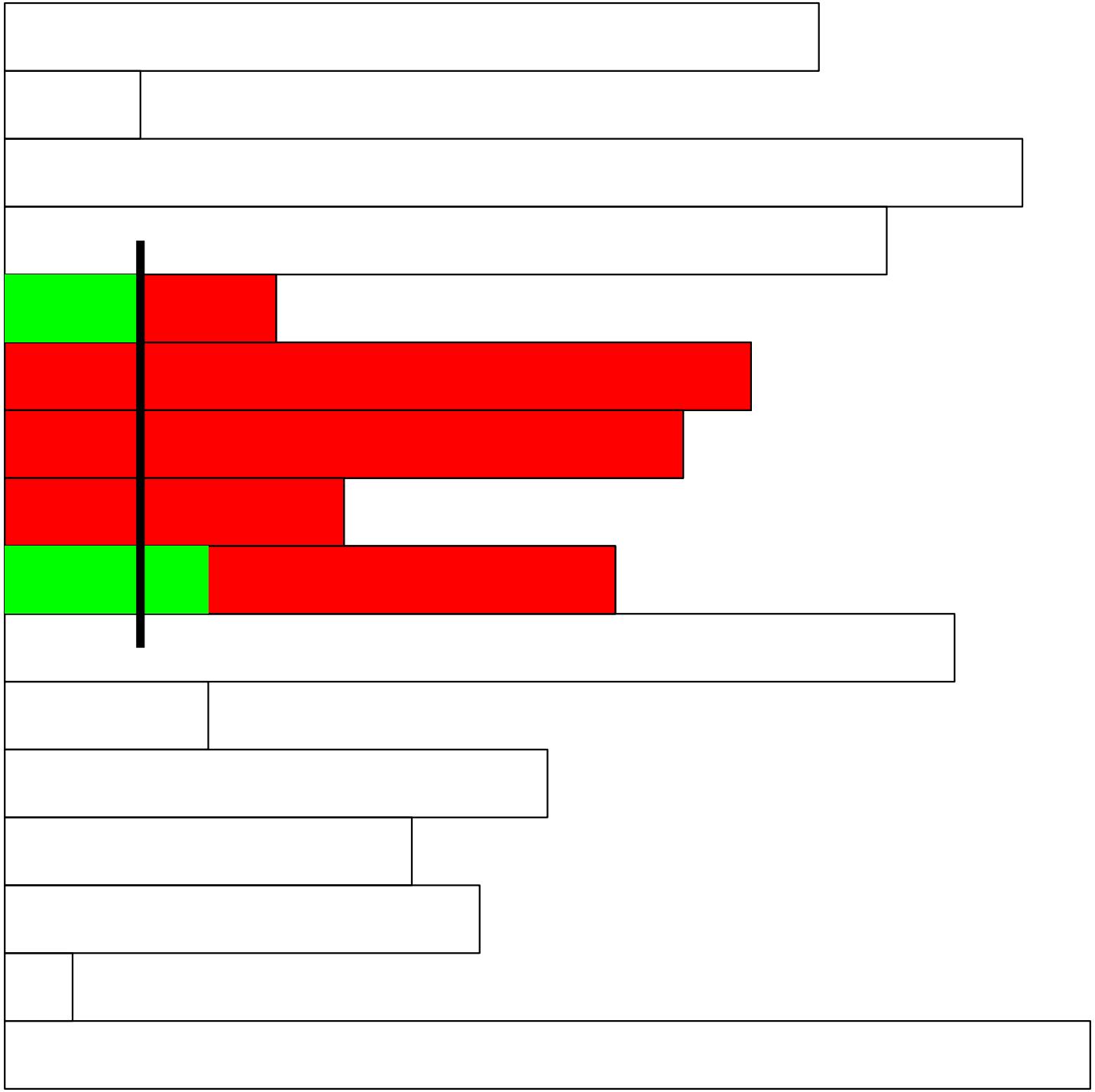
All lcp can be computed in amortized constant time per entry.

Proof: next problem set.




L
M
R





Invariant

We maintain a range $[L, R]$ such that the answer is somewhere inside, and we know the longest common prefix of $SA[L]$ and p, and $SA[R]$ and p.

We choose $M \in (L, R)$. Of course we know that the longest common prefix of $SA[M]$ and p is at least as long as the minimum of the two known prefixes, but we can notice even more.
Let ℓ be the longest common prefix of $SA[L]$ and p, and r be the longest common prefix of $SA[R]$ and p. Assume that $\ell \leq r$, the situation is symmetric so the other case is very similar.

Invariant

We maintain a range $[L, R]$ such that the answer is somewhere inside, and we know the longest common prefix of $SA[L]$ and p, and $SA[R]$ and p.

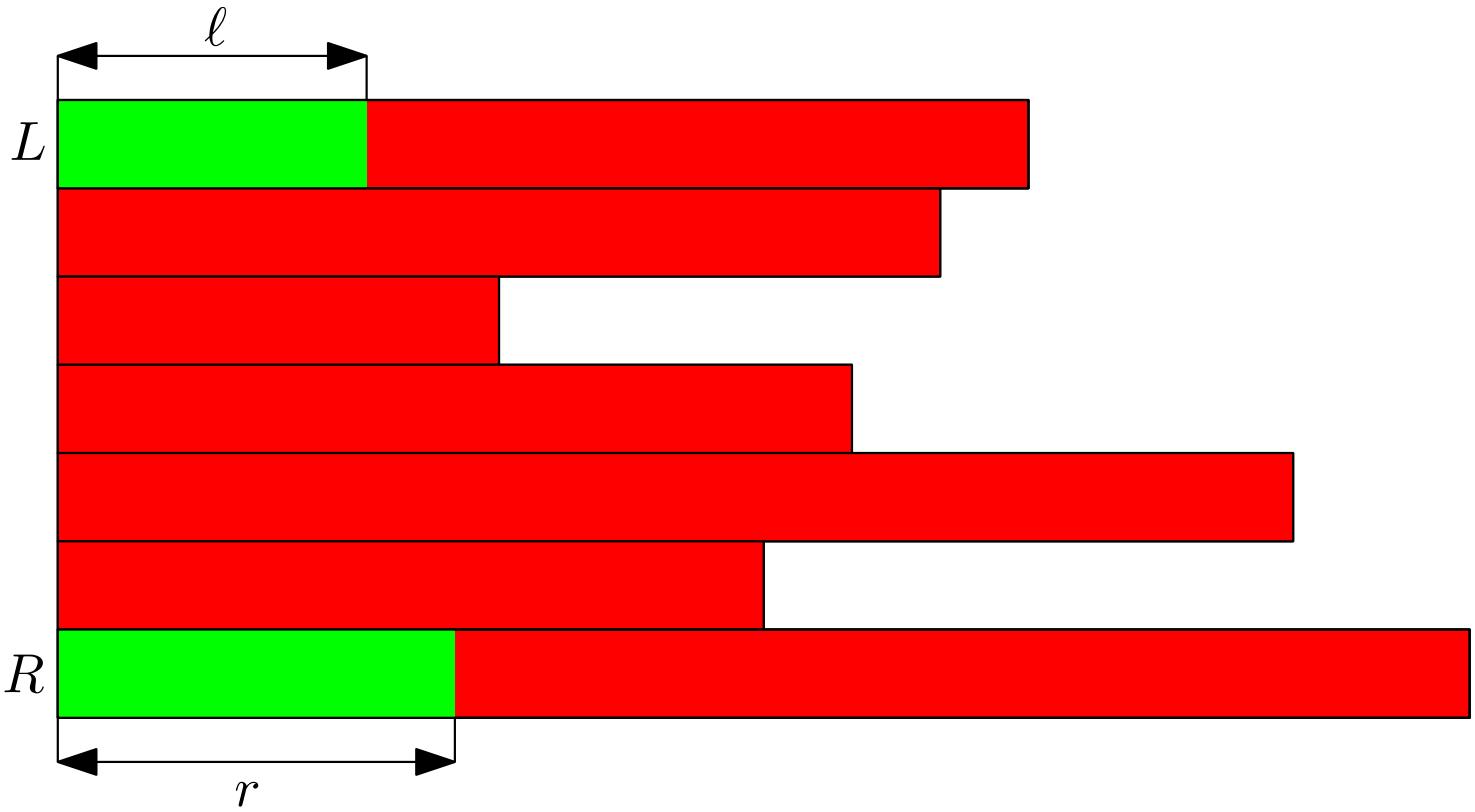
We choose $M \in (L, R)$. Of course we know that the longest common prefix of $SA[M]$ and p is at least as long as the minimum of the two known prefixes, but we can notice even more.

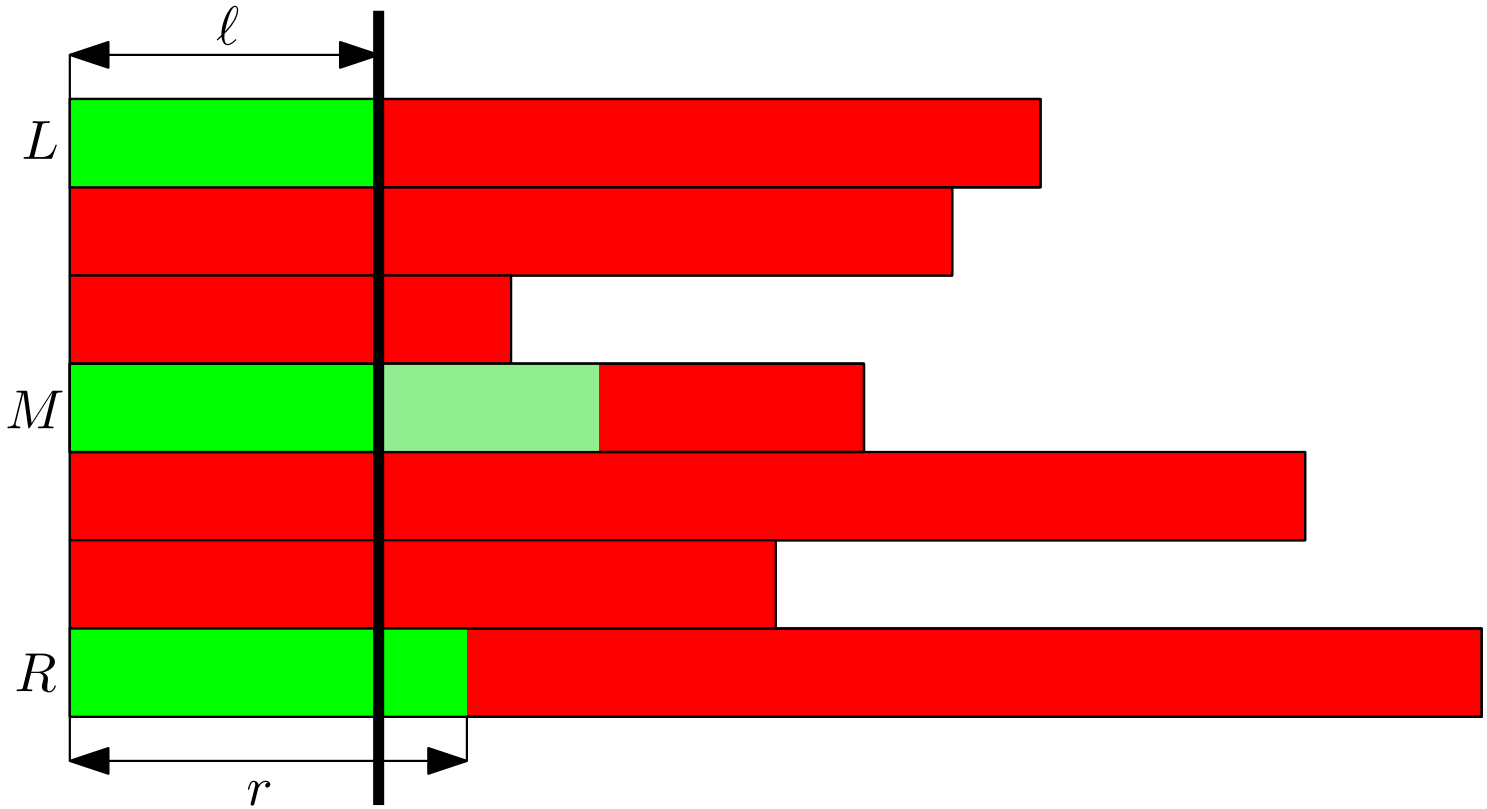
Let ℓ be the longest common prefix of $SA[L]$ and p, and r be the longest common prefix of $SA[R]$ and p. Assume that $\ell \leq r$, the situation is symmetric so the other case is very similar.

Invariant

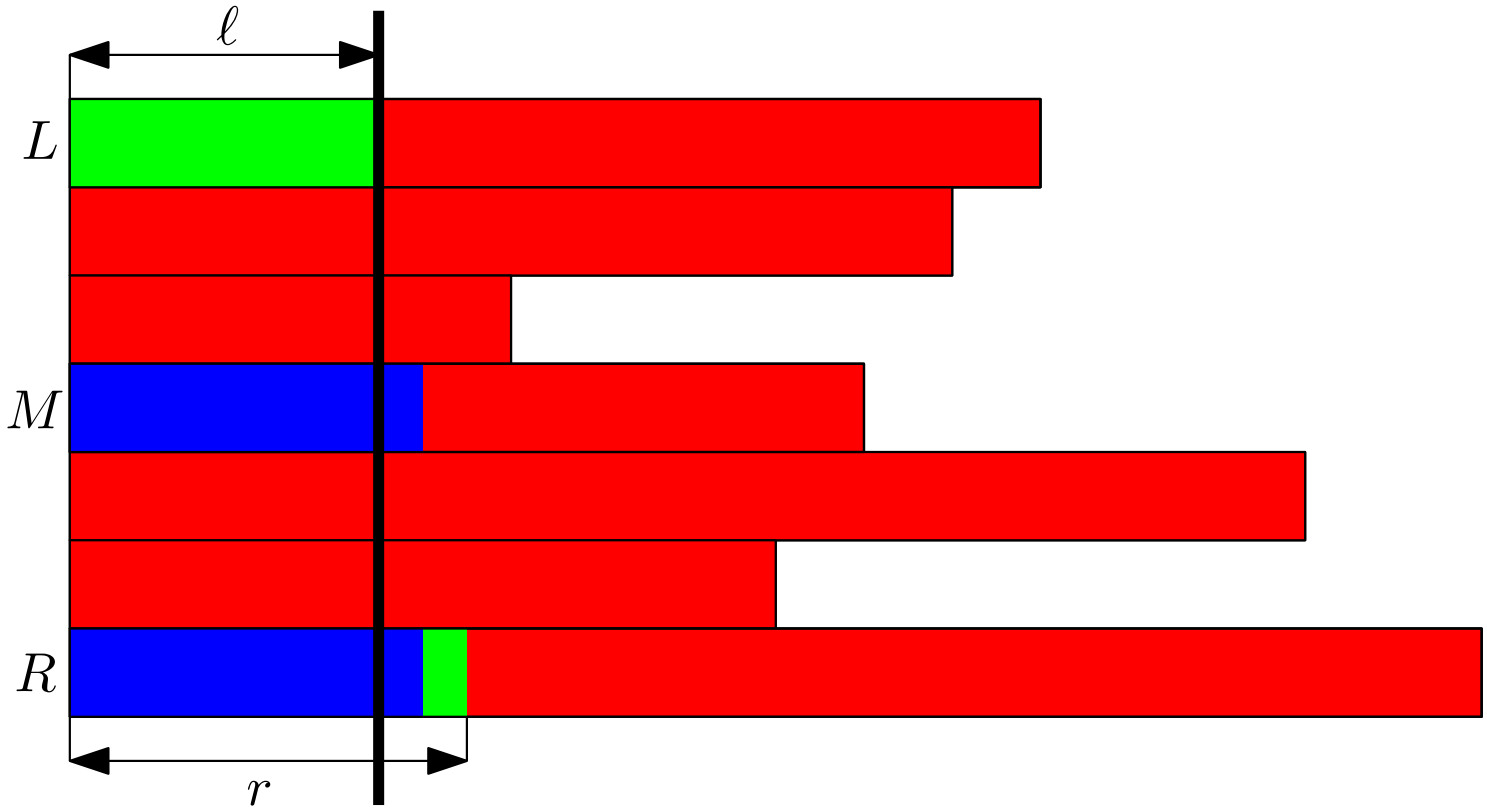
We maintain a range $[L, R]$ such that the answer is somewhere inside, and we know the longest common prefix of $SA[L]$ and p, and $SA[R]$ and p.

We choose $M \in (L, R)$. Of course we know that the longest common prefix of $SA[M]$ and p is at least as long as the minimum of the two known prefixes, but we can notice even more.
Let ℓ be the longest common prefix of $SA[L]$ and p, and r be the longest common prefix of $SA[R]$ and p. Assume that $\ell \leq r$, the situation is symmetric so the other case is very similar.

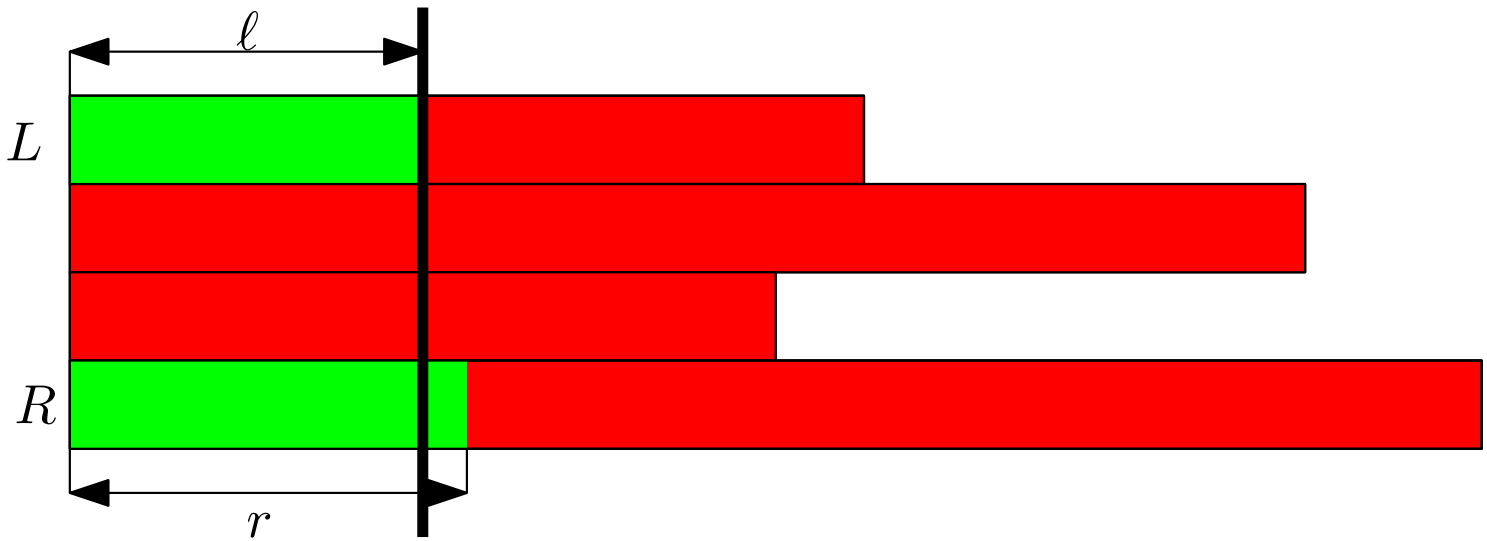

ℓ
L
R
r



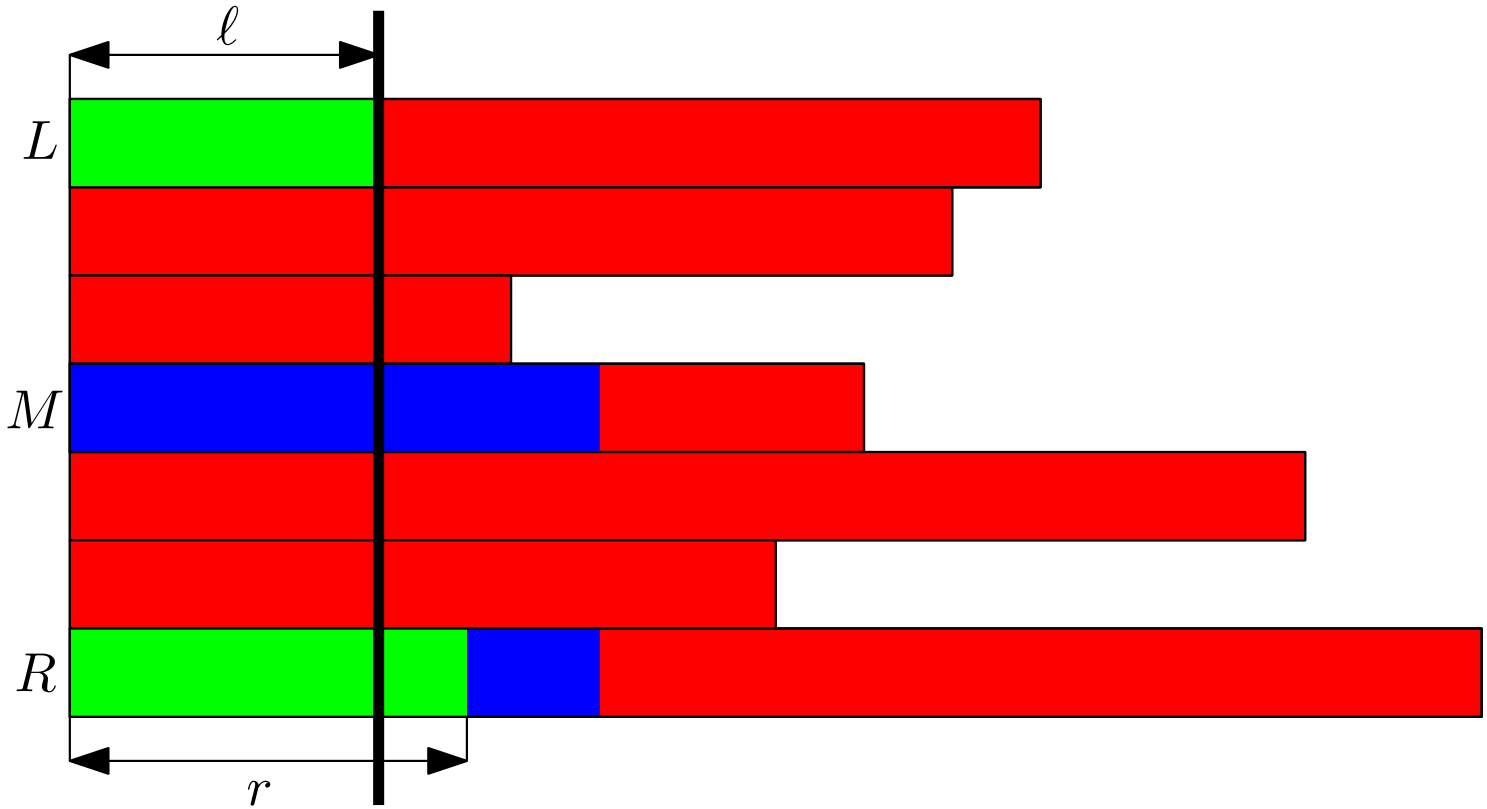
Look at $\mathrm{lcp}(SA[M], SA[R])$.



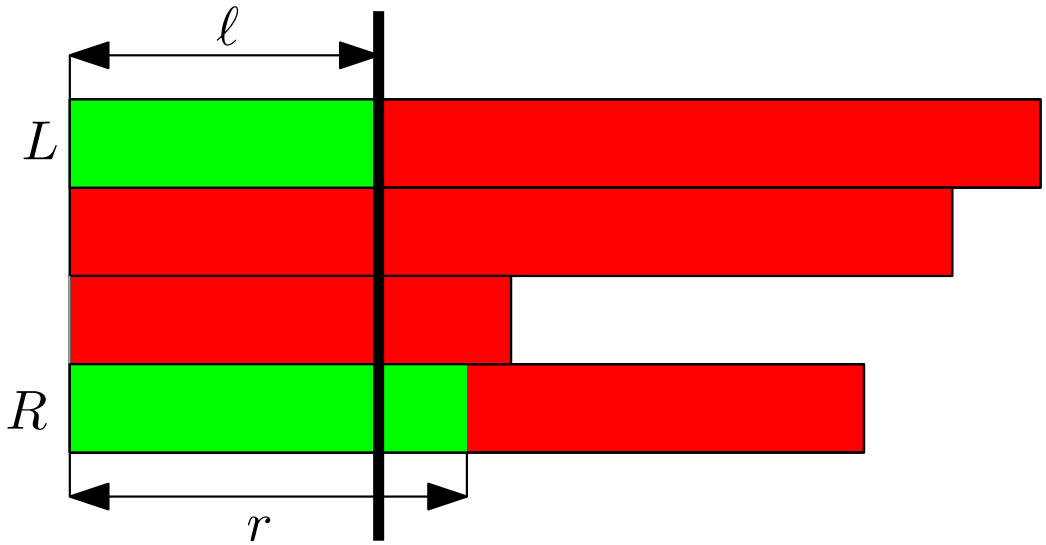
If $\mathrm{lcp}(SA[M], SA[R]) < r$, set $L = M$ and $\ell = \mathrm{lcp}(SA[M], SA[R])$.



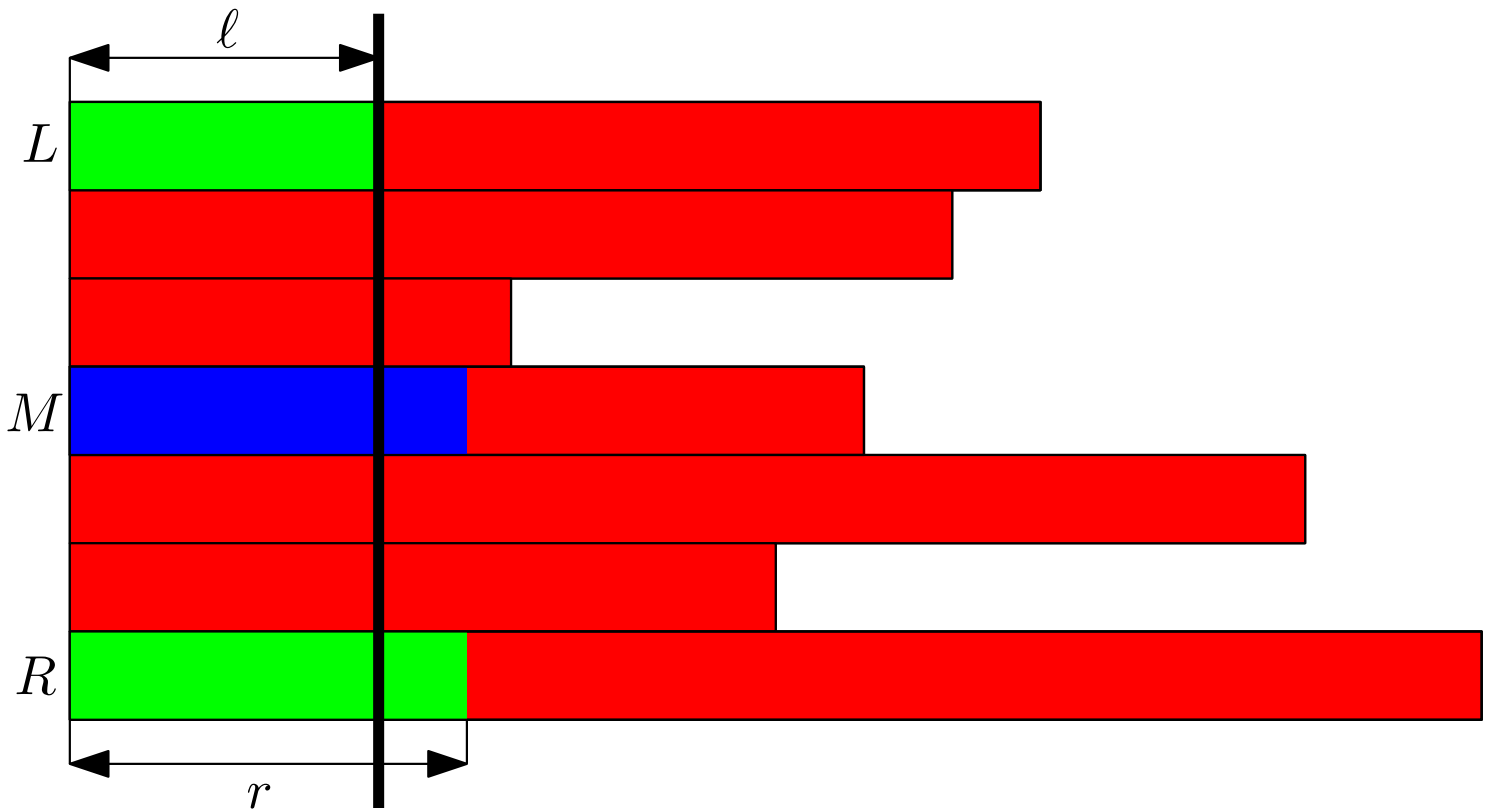
If $\mathrm{lcp}(SA[M], SA[R]) < r$, set $L = M$ and $\ell = \mathrm{lcp}(SA[M], SA[R])$.



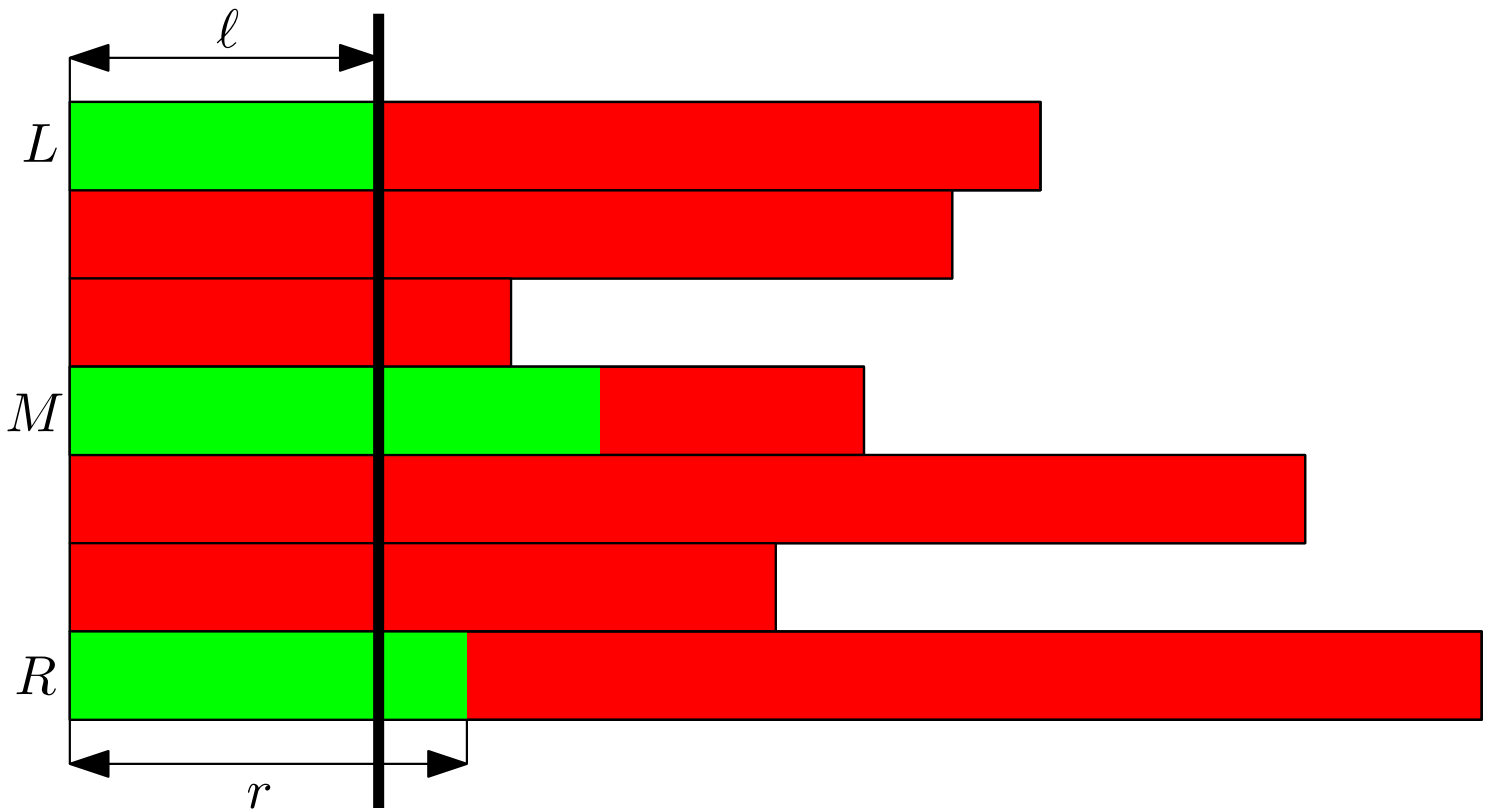
If $\mathrm{lcp}(SA[M], SA[R]) > r$, set $R = M$ and keep old ℓ and r.



If $\mathrm{lcp}(SA[M], SA[R]) > r$, set $R = M$ and keep old ℓ and r.



If $\mathrm{lcp}(SA[M], SA[R]) = r$, compute the longest common prefix of $SA[M]$ and p, but start from the r-th character. Depending on the next character set $L = M$ or $R = M$.



If $\mathrm{lcp}(SA[M], SA[R]) = r$, compute the longest common prefix of $SA[M]$ and p, but start from the r-th character. Depending on the next character set $L = M$ or $R = M$.

Let's look again at the last case. Say that the longest common prefix of $SA[M]$ and p be m. We have two cases:

- the next character of $SA[M]$ is less than $p[m+1]$, then we set $L = M$ and $\ell = m$,
- the next character of $SA[M]$ is greater than $p[m+1]$, then we set $R = M$ and $r = m$.

In both cases we spent just $\mathcal{O}(m - r + 1)$ time computing the longest common prefix.

The value of $\ell + r$ doesn't decrease.

It follows that the sum of $\mathcal{O}(m - r)$ over all steps of the procedure is just $\mathcal{O}(m)$ in the worst possible case. We additionally spent $\mathcal{O}(1)$ time per step to look at $SA[M]$, hence the total complexity is $\mathcal{O}(m + \log n)$.

Let's look again at the last case. Say that the longest common prefix of $SA[M]$ and p be m. We have two cases:

- the next character of $SA[M]$ is less than $p[m+1]$, then we set $L = M$ and $\ell = m$,
- the next character of $SA[M]$ is greater than $p[m+1]$, then we set $R = M$ and $r = m$.

In both cases we spent just $\mathcal{O}(m - r + 1)$ time computing the longest common prefix.

The value of $\ell + r$ doesn't decrease.

It follows that the sum of $\mathcal{O}(m - r)$ over all steps of the procedure is just $\mathcal{O}(m)$ in the worst possible case. We additionally spent $\mathcal{O}(1)$ time per step to look at $SA[M]$, hence the total complexity is $\mathcal{O}(m + \log n)$.

Let's look again at the last case. Say that the longest common prefix of $SA[M]$ and p be m. We have two cases:

- the next character of $SA[M]$ is less than $p[m+1]$, then we set $L = M$ and $\ell = m$,
- the next character of $SA[M]$ is greater than $p[m+1]$, then we set $R = M$ and $r = m$.

In both cases we spent just $\mathcal{O}(m - r + 1)$ time computing the longest common prefix.

The value of $\ell + r$ doesn't decrease.

It follows that the sum of $\mathcal{O}(m - r)$ over all steps of the procedure is just $\mathcal{O}(m)$ in the worst possible case. We additionally spent $\mathcal{O}(1)$ time per step to look at $SA[M]$, hence the total complexity is $\mathcal{O}(m + \log n)$.

Let's look again at the last case. Say that the longest common prefix of $SA[M]$ and p be m. We have two cases:

- the next character of $SA[M]$ is less than $p[m+1]$, then we set $L = M$ and $\ell = m$,
- the next character of $SA[M]$ is greater than $p[m+1]$, then we set $R = M$ and $r = m$.

In both cases we spent just $\mathcal{O}(m - r + 1)$ time computing the longest common prefix.

The value of $\ell + r$ doesn't decrease.

It follows that the sum of $\mathcal{O}(m - r)$ over all steps of the procedure is just $\mathcal{O}(m)$ in the worst possible case. We additionally spent $\mathcal{O}(1)$ time per step to look at $SA[M]$, hence the total complexity is $\mathcal{O}(m + \log n)$.

Let's look again at the last case. Say that the longest common prefix of $SA[M]$ and p be m. We have two cases:

- the next character of $SA[M]$ is less than $p[m+1]$, then we set $L = M$ and $\ell = m$,
- the next character of $SA[M]$ is greater than $p[m+1]$, then we set $R = M$ and $r = m$.

In both cases we spent just $\mathcal{O}(m - r + 1)$ time computing the longest common prefix.

The value of $\ell + r$ doesn't decrease.

It follows that the sum of $\mathcal{O}(m - r)$ over all steps of the procedure is just $\mathcal{O}(m)$ in the worst possible case. We additionally spent $\mathcal{O}(1)$ time per step to look at $SA[M]$, hence the total complexity is $\mathcal{O}(m + \log n)$.

Let's look again at the last case. Say that the longest common prefix of $SA[M]$ and p be m. We have two cases:

- the next character of $SA[M]$ is less than $p[m+1]$, then we set $L = M$ and $\ell = m$,
- the next character of $SA[M]$ is greater than $p[m+1]$, then we set $R = M$ and $r = m$.

In both cases we spent just $\mathcal{O}(m - r + 1)$ time computing the longest common prefix.

The value of $\ell + r$ doesn't decrease.

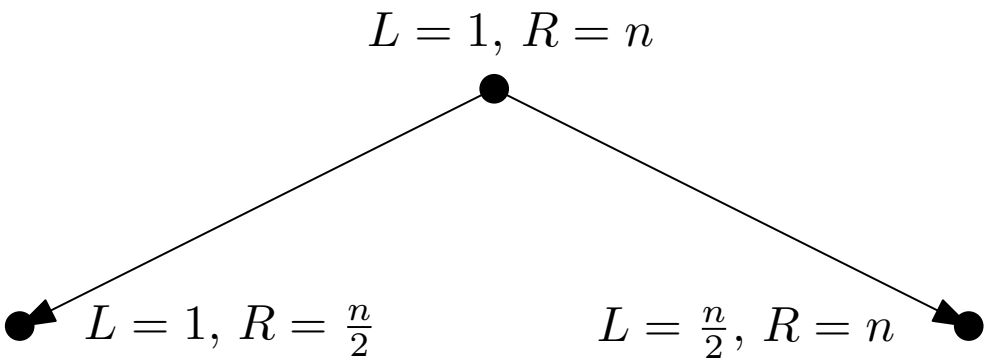
It follows that the sum of $\mathcal{O}(m - r)$ over all steps of the procedure is just $\mathcal{O}(m)$ in the worst possible case. We additionally spent $\mathcal{O}(1)$ time per step to look at $SA[M]$, hence the total complexity is $\mathcal{O}(m + \log n)$.

Recall that we assumed that computing any $\mathrm{lcp}(i, j)$ takes constant time. While it can be done (as we will soon see), that's an overkill. Do we really need to compute any such value?

$$L = 1,\ R = n$$

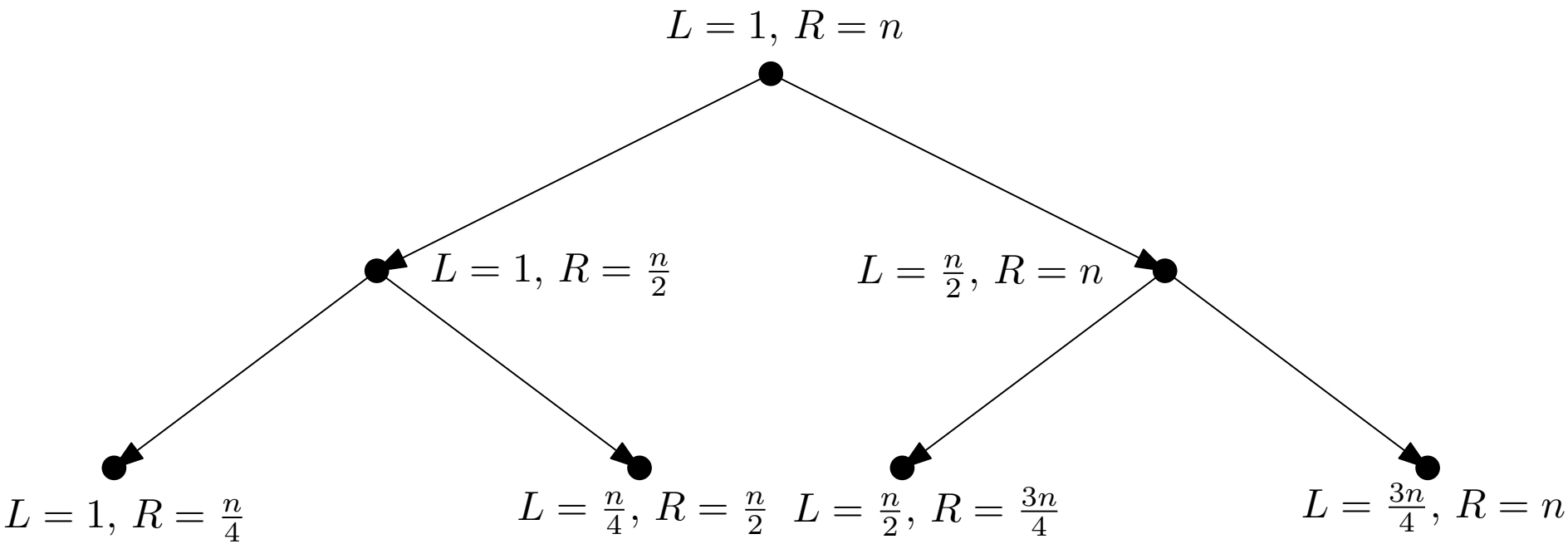
Each node of the recursion tree generates just two values $\mathrm{lcp}(SA[L], SA[M])$ and $\mathrm{lcp}(SA[M], SA[R])$ to be computed. Hence we have just $\mathcal{O}(n)$ values in total!

Recall that we assumed that computing any $\mathrm{lcp}(i, j)$ takes constant time. While it can be done (as we will soon see), that's an overkill. Do we really need to compute any such value?



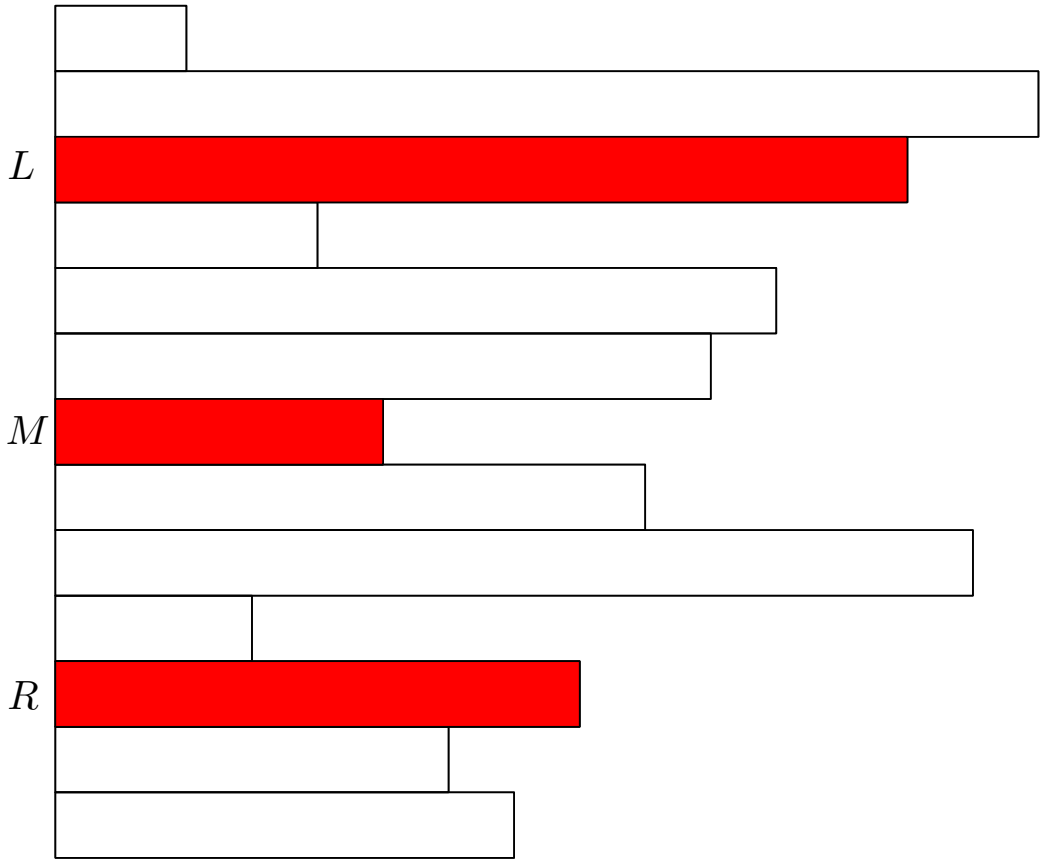
Each node of the recursion tree generates just two values $\mathrm{lcp}(SA[L], SA[M])$ and $\mathrm{lcp}(SA[M], SA[R])$ to be computed. Hence we have just $\mathcal{O}(n)$ values in total!

Recall that we assumed that computing any $\mathrm{lcp}(i, j)$ takes constant time. While it can be done (as we will soon see), that's an overkill. Do we really need to compute any such value?



Each node of the recursion tree generates just two values $\mathrm{lcp}(SA[L], SA[M])$ and $\mathrm{lcp}(SA[M], SA[R])$ to be computed. Hence we have just $\mathcal{O}(n)$ values in total!

All those values can be actually computed in $\mathcal{O}(n)$ time in a bottom-top manner.



Lemma

$\mathrm{lcp}(SA[L], SA[R]) = \min(\mathrm{lcp}(SA[L], SA[M]), \mathrm{lcp}(SA[M], SA[R]))$

Proof: see the blackboard.